Algoritmos y Ejercicios

Módulo 2-Grupo Los Pi

2026-04-01

R Markdown

Este es un documento de R Markdown. Markdown es una sintaxis de formato simple para crear documentos HTML, PDF y MS Word. Para más detalles sobre el uso de R Markdown, consulte http://rmarkdown.rstudio.com.

Cuando haces clic en el botón **Knit**, se generará un documento que incluye tanto el contenido como la salida de cualquier fragmento de código R incrustado dentro del documento. Puedes incrustar un fragmento de código R así:

Primeros comandos de R

R es un lenguaje muy similar a Matlab y permite el trabajo con matrices.

```
A=38
A=40
B=60
C=B-A
```

```
Y=4
alpha<-30
```

Uso de datasets o base de datos internos de R

Usando el comando **data** en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas específicas si escribimos **$** despues de el comando de información específico y elegimos la columna de datos deseada. También existe el comando **summary**, que nos otorga una estadística descriptiva básica sobre la información.

```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

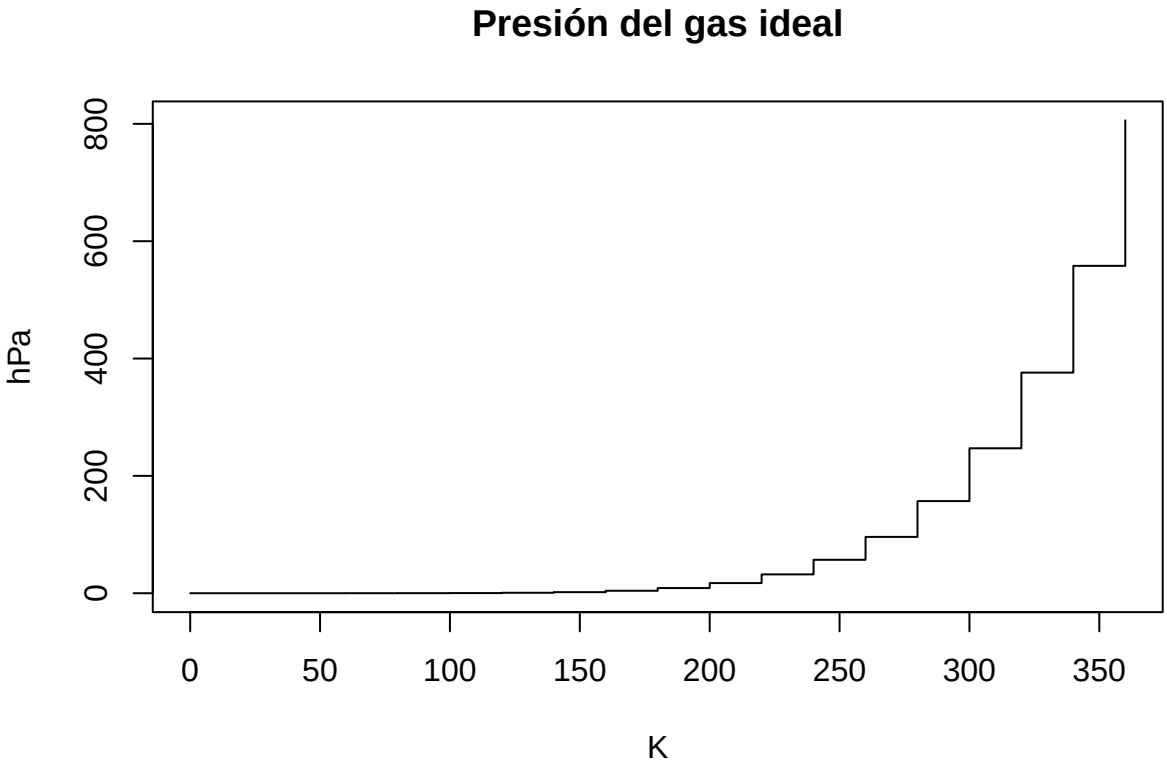
```
cars$speed
```

```
##  [1]  4  4  7  7  8  9 10 10 10 11 11 12 12 12 12 13 13 13 13 14 14 14 14 15 15
## [26] 15 16 16 17 17 17 18 18 18 18 19 19 19 20 20 20 20 20 22 23 24 24 24 24 25
```

Comando plot

Es un comando que permite gráfica cualquier fuente de datos que le asignemos. Si escrbimos **?plot** en la consola, podemos ver una guía de comandos para editar nuestro gráfico (titulos, nombrar variables, etc).

```
plot(pressure,type="s",main="Presión del gas ideal",ylab="hPa",xlab="K")
```



Funciones estadísticas

Podemos usar el comando **rnorm**, que genera datos aleatorios pero con la opción de seguir una Distribución Normal (campana de Gauss). Para los siguientes valores: **n=350** (tamaño de la muestra), con un **media=22** (valor promedio) y **desviación estándar=5** (dispersión).

```
z1<-rnorm(350,22,5)
z1
```

```
##   [1] 27.466034 20.292999 15.651132 23.170227 22.228796 24.137581 20.902559
##   [8] 17.483458 19.802169 25.165236 22.716783 17.672458 24.214851 22.445778
##  [15] 18.942454 19.133458 25.344289 15.135322 25.394216 15.785958 20.801048
##  [22] 36.487838 20.935566 24.313463 28.707907 22.904495 21.612082 22.427741
##  [29] 17.704809 17.287397 20.403066 25.193970 21.329840 12.326637 21.435949
##  [36] 28.784855 29.561613 22.162870 22.760646 15.451496 28.373990 28.219656
##  [43] 22.331447 18.380976 28.280376 32.171696 16.108274 23.540362 21.722537
##  [50] 30.825613 24.499228 24.849657 18.796811 17.774096 15.895141 26.989774
##  [57] 25.812092 26.176254 31.313513 25.335158 18.668384 23.153434 23.354362
##  [64] 20.412913 19.541010 19.184761 24.414242 21.360095 17.633657 21.131777
##  [71] 12.746015 14.859161 25.762745 11.575089 16.468378  4.903643 17.315656
##  [78] 25.272187 15.449980 27.833330 18.516650 19.761442 17.054211 21.948407
##  [85] 21.350905 12.954857 24.072018 15.071824 32.469223 17.907324 17.682120
##  [92] 17.252827 31.594310 31.072214 24.361703 31.593493 22.713791 13.260647
##  [99] 15.255273 21.243191 15.458433 19.687991 22.427293 25.582448 13.321245
```

```
## [106] 19.504403 14.868221 19.399009 27.950156 25.981356 18.341644 22.779162
## [113] 21.512001 16.544322 19.519481 15.489616 22.705953 23.592898 23.715725
## [120] 32.308583 19.505855 21.267186 24.228814 22.459405 23.446190 26.628549
## [127] 23.313764 27.330576 18.817616 19.071841 19.190287 25.014813 17.119756
## [134] 15.514597 21.307088 22.238914 24.466213 21.062671 29.792924 24.164399
## [141] 31.575218 18.895100 15.619017 27.235461 23.578029 17.778113 20.043340
## [148] 19.357164 21.650200 24.645074 16.733940 21.070567 23.024273 21.781208
## [155] 22.172774 18.643510 30.663220 20.545477 31.467178 25.307117 25.202778
## [162] 21.723157 22.337051 15.938775 23.333388  9.316800 31.913762 12.959656
## [169] 23.173455 16.665939 19.745180 28.068057 19.859867 23.736831 22.017113
## [176] 15.702003 22.260402 19.900055 21.090022 20.747807 25.997794 30.749259
## [183] 16.558274 29.118346 16.933713 23.399815 16.629242 25.195029 27.172431
## [190] 20.664624 27.318878 15.645839 16.404405 30.317166 24.306006 25.241836
## [197] 27.494173 15.726677 18.929982 25.565630 23.262472 23.087462 19.441995
## [204] 13.305270 20.156834 20.029842 17.787137 19.482661 24.171006 24.886645
## [211] 10.960583 21.021482 19.556941 22.422330 10.319855 32.917928 28.036412
## [218] 30.194751 14.509777 21.079949 18.342731 22.668590 23.078131 25.840883
## [225] 27.963893 27.778316 24.518508 27.873516 26.680679 24.914658 25.853373
## [232] 25.674687 17.991587 13.473916 30.536484 18.374455 24.824581 25.582328
## [239] 26.402826 18.746188 21.184990 24.603835 27.486279 21.714300 23.624783
## [246] 13.210427 20.867022 22.970836 16.479445 29.083820 26.905992 23.594443
## [253] 27.230667 19.217205 19.449383 29.314489 28.388749 20.194008 23.569063
## [260] 27.668429 25.465763 16.127321 20.271137 20.866461 19.845234 28.886612
## [267] 20.686218 22.773654 17.678452 25.467879 15.496436 22.379626 23.468288
## [274] 32.410677 22.158655 25.681984 19.688704 25.562452 19.891053 12.602691
## [281] 19.461767 20.352307 34.434017 24.602549 30.104988 17.369986 24.913612
## [288] 21.563373  9.004222 26.567326 20.207616 23.275089 17.221153 19.924054
## [295] 21.496743 27.710774 22.783426 29.474271 26.071913 18.116900 20.864504
## [302] 21.014327 16.028306 22.454419 15.991911 27.501412 25.717167 14.632555
## [309] 23.647165 19.168614 19.419220 19.966184 22.123090 23.770824 22.015731
## [316] 24.539383 26.141663 24.793342 20.140806 24.203041 16.371843 18.585075
## [323] 26.217084 23.192764 18.071596 26.589482 23.238935 19.876339 19.759579
## [330] 13.393740 24.245996 20.022292 19.558370  8.325281 16.700416 18.758679
## [337] 20.703039 23.195768 33.245344 17.424492 30.716069 17.320114 27.180268
## [344] 19.822188 17.546492 21.607855 18.697022 19.057293 22.025016 15.231286
```

```r
w1<-length(z1)
w1
```

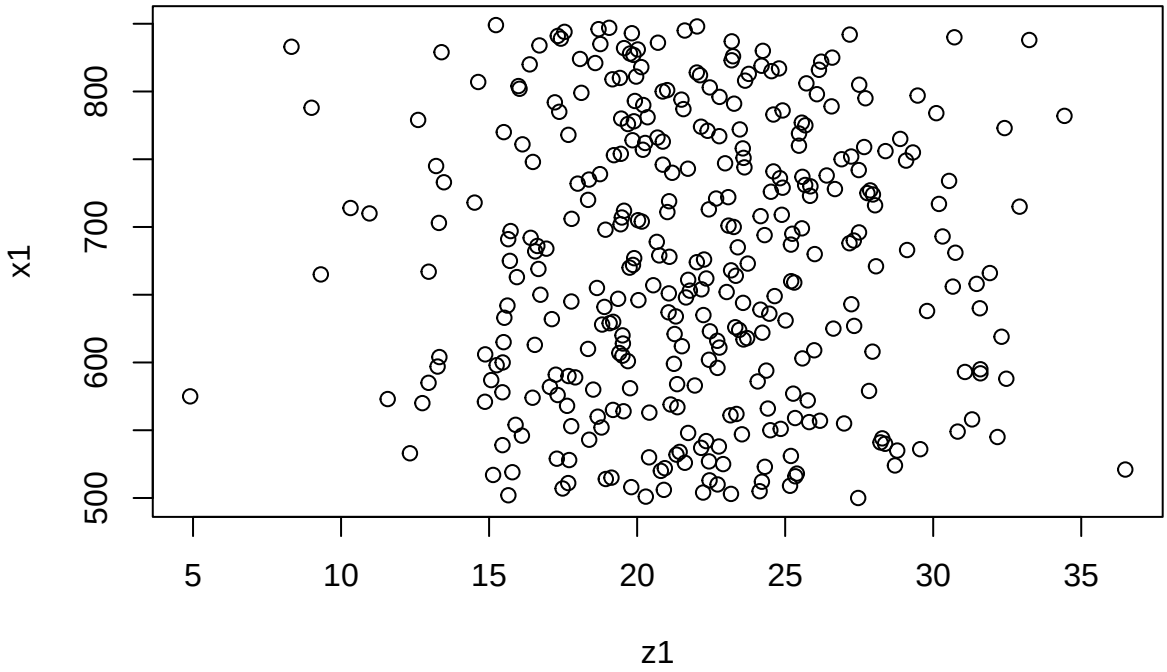
```
## [1] 350
```

```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
```

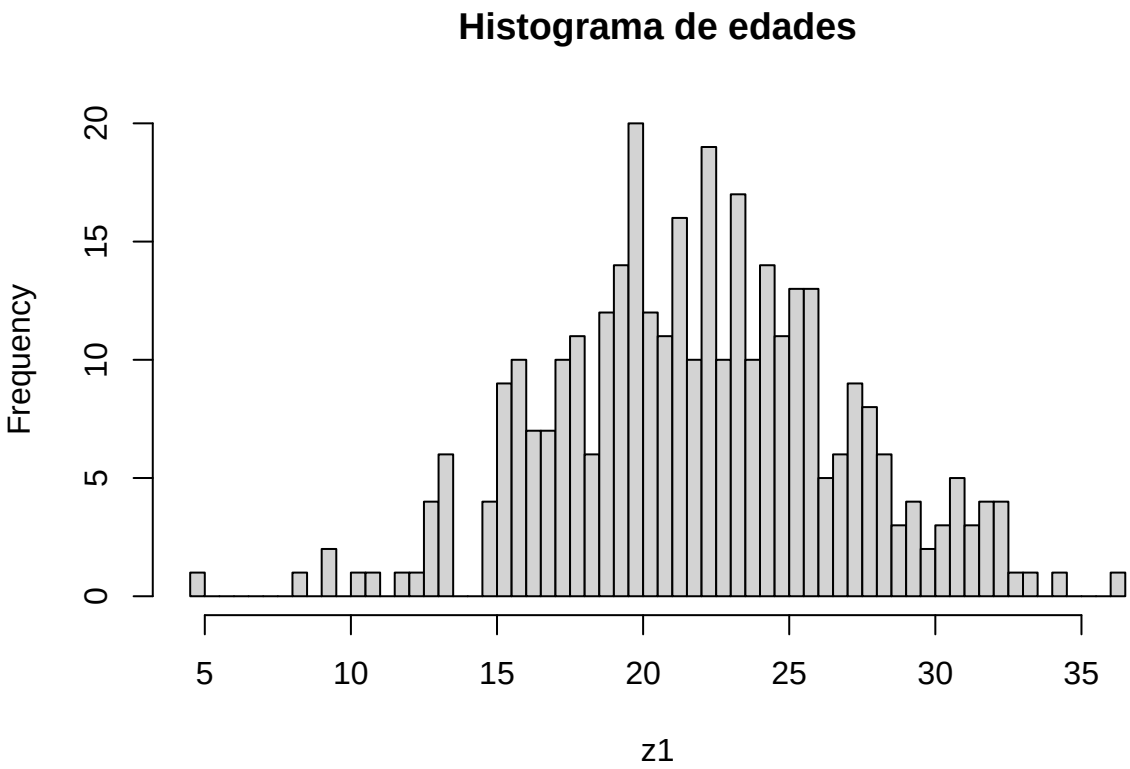
```
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1,x1)
```



Para generar un histograma: comando **hist**.

```r
hist(z1,main="Histograma de edades",breaks = 50)
```

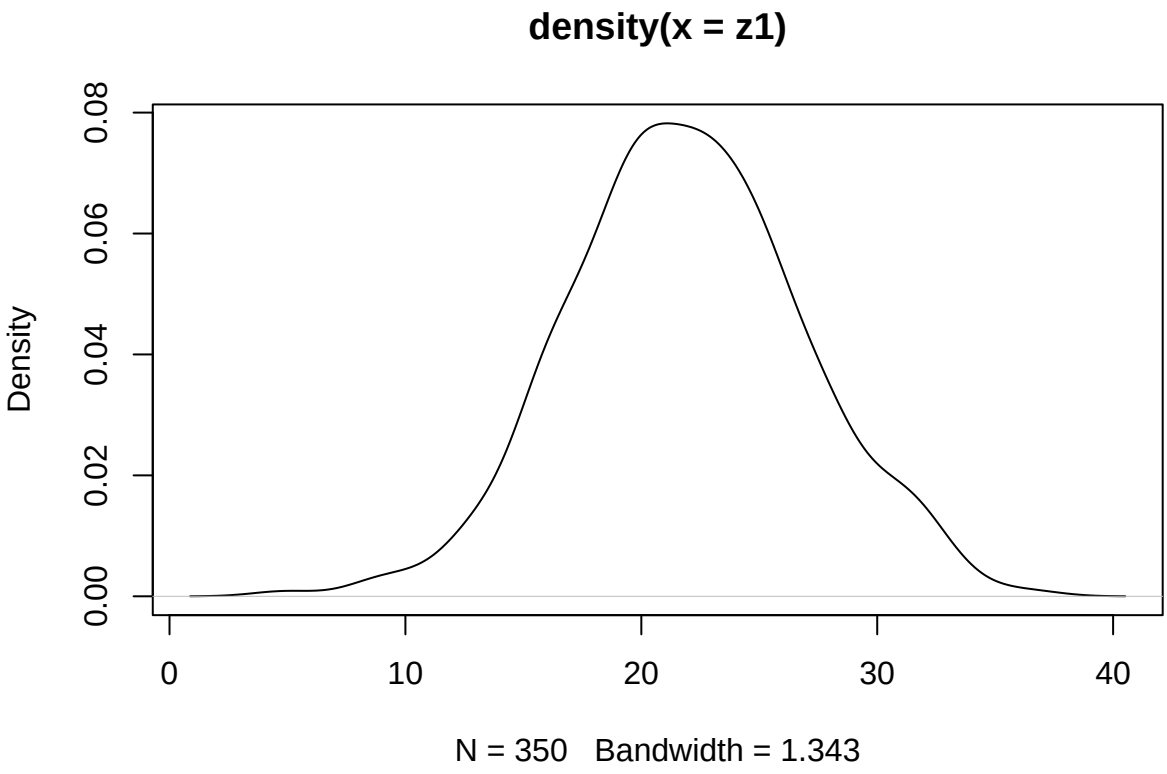


Para calcular la densidad de probabilidad: comando **density**.

```
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.343
##
##        x                 y
##  Min.   : 0.8755   Min.   :9.526e-06
##  1st Qu.:10.7856   1st Qu.:1.174e-03
##  Median :20.6957   Median :1.267e-02
##  Mean   :20.6957   Mean   :2.518e-02
##  3rd Qu.:30.6059   3rd Qu.:4.774e-02
##  Max.   :40.5160   Max.   :7.822e-02
```

```
plot(density(z1))
```



Ejercicios Algoritmos

Ejercicio 1.1

Las últimas 3 cifras de mi DNI son 525, crear una variable que tenga ese número.

```
DNI=525
```

Ejercicio 1.2

Crear un vector que tenga todos los números desde el 1 hasta el 525.

```
Secuencia_DNI<-(1:525)
Secuencia_DNI
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
```

```
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525
```

Ejercicio 2

Calcular la suma de todos los valores del vector Secuencia_DNI.

```
Total<-0
Valor_final<-length(Secuencia_DNI)
for(i in 1:Valor_final){Total<-Total+i}
Total
```

```
## [1] 138075
```

Ejercicio 3.1

Repetir el ejercicio anterior pero en Python.

```
suma=0
for i in range(1,526):
  suma+=1
print(suma)
```

```
## 525
```

Ejercicio 3.2

¿Cuánto tarda en correr el código del ejercicio 2?

```
inicio<-Sys.time()
Total<-0
Valor_final<-100000*length(Secuencia_DNI)
for (i in 1:Valor_final){Total<-Total+1}
Total
```

```
## [1] 52500000
```

```
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.8041792 secs
```

Ejercicio 3.3

Repetir el ejercicio anterior utilizando la cabeza de Gauss.

```r
inicio<-Sys.time()
n<-10*length(Secuencia_DNI)
total<-n*(n+1)/2
total
```

```
## [1] 13783875
```

```r
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.002158165 secs
```

Ejercicio 4:

Generar una secuencia de 2 en 2 hasta 100000.

```r
inicio<-Sys.time()
A<-0
for (i in 1:50000){A[i]<-i*2}
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.02334404 secs
```

```r
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

Secuencia con función integrada de R (comando seq->más eficiente)

```r
inicio<-Sys.time()
A<-seq(2,100000,by=2)
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.001305103 secs
```

```r
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

Implementación de una Serie de Fibonacci.

¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

Ejercicio 5-Mediante función integrada

```r
fib<-c(0,1)
while(tail(fib,1)<1e6) {fib<-c(fib, fib[length(fib)]+fib[length(fib)-1])}
fib
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
```

```
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```r
print(paste('Iteraciones necesarias:',length(fib)))
```

```
## [1] "Iteraciones necesarias: 32"
```

Ejercicio 6-Mediante definición matemática (Método Iterativo)

```r
inicio<-Sys.time()
f0<-0
f1<-1
contador<-0
while (f1<=1000000){
  f2<-f1+f0
  div<-f2/f1
  f0<-f1
  f1<-f2
  contador<-contador+1}
print(paste('Iteraciones necesarias:', contador))
```

```
## [1] "Iteraciones necesarias: 30"
```

```r
print(paste('Valor alcanzado:', f1))
```

```
## [1] "Valor alcanzado: 1346269"
```

```r
print(paste('Cociente:', div))
```

```
## [1] "Cociente: 1.61803398875054"
```

```r
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.00666523 secs
```

Ejercicio 7:

Ordenación de un vector por método burbuja. Burbuja vs Sort.

```r
library(microbenchmark)
x<-sample(1:20000, 20000)
burbuja<-function(x){
  n<-length(x)
  for (j in 1:(n-1)){
    for (i in 1:(n-j)){
      if (x[i]>x[i+1]){
        temp<-x[i]
        x[i]<-x[i+1]
        x[i+1]<-temp}}}
  return(x)}
resultado<-microbenchmark(
  burbuja=burbuja(x),
  sort_R=sort(x),
  times=3)
summary(resultado)
```

```
##      expr         min          lq         mean      median          uq
```

```
## 1 burbuja 21684863.40 21891112.01 2.223794e+07 22097360.61 22514485.52
## 2  sort_R      181.23      241.49 3.129033e+02      301.75      378.74
##           max neval
## 1 22931610.42     3
## 2      455.73     3
```

Conclusión

Al analizar ambos métodos mediante microbenchmarks, se nota que `sort()` tiene un rendimiento mucho mejor que el algoritmo de burbuja. Esto ocurre porque el método burbuja realiza comparaciones repetidas entre pares de elementos, lo que lo vuelve ineficiente, mientras que `sort()` emplea algoritmos internos más optimizados. Cuando se trabaja con un arreglo de 20.000 elementos, la diferencia en el tiempo de ejecución es considerable, evidenciando que el algoritmo de burbuja no es una buena opción para manejar conjuntos de datos grandes.

Ejercicio 8

Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×10^{10} y verifica cuál de los dos es más eficiente.

```r
suma_iterativa<-function(n){
  total<-0
  for (i in 1:n){total<-total+i}
  return(total)}
suma_gauss<-function(n){return(n*(n+1)/2)}
library(microbenchmark)
n<-1e6
resultado<-microbenchmark(
  iterativo=suma_iterativa(n),
  gauss=suma_gauss(n),
  times=5)
resultado
```

1er Forma con Microbenchmark

```
## Unit: microseconds
##       expr      min       lq      mean   median       uq      max neval
##  iterativo 15200.05 15572.00 16565.768 17040.63 17240.56 17775.60     5
##      gauss     3.53     4.85   421.066     6.48    17.26  2073.21     5
```

```r
summary(resultado)
```

```
##        expr      min       lq      mean   median       uq      max neval
## 1 iterativo 15200.05 15572.00 16565.768 17040.63 17240.56 17775.60     5
## 2     gauss     3.53     4.85   421.066     6.48    17.26  2073.21     5
```

```r
suma_iterativa(n)
```

```
## [1] 500000500000
```

```r
suma_gauss(n)
```

```
## [1] 500000500000
```

```r
n<-1e6
inicio<-Sys.time()
```

```r
  suma<-0
  for (i in 1:n){suma<-suma+i}
final<-Sys.time()
final-inicio
```

2da forma

```
## Time difference of 0.01881123 secs
```

```r
suma
```

```
## [1] 500000500000
```

```r
inicio<-Sys.time()
suma2<-n*(n+1)/2
final<-Sys.time()
final-inicio
```

```
## Time difference of 0.0008802414 secs
```

```r
suma2
```

```
## [1] 500000500000
```

Conclusión

El método iterativo tarda más y, por lo tanto, es menos eficiente, ya que debe realizar una suma por cada elemento. En cambio, el método de Gauss es más rápido y eficiente porque utiliza una fórmula directa, reduciendo la cantidad de operaciones, el uso de recursos y el tiempo de ejecución.

Por esta razón, el método de Gauss resulta mucho más eficiente, sobre todo cuando n es grande. Para este caso, donde $n = 1 \times 10^{10}$, el método iterativo se vuelve impracticable, mientras que el de Gauss lo resuelve de inmediato.

No obstante, para lograr ejecutar el programa sin problemas, se optó por trabajar con $n = 1 \times 10^6$, ya que utilizar un valor de orden 10^10 que con el método iterativo impedía completar la ejecución.